

Efficient Neural Ranking using Forward Indexes

Jurek Leonhardt
L3S Research Center
Hannover, Germany
leonhardt@L3S.de

Koustav Rudra*
Indian Institute of Technology
(Indian School of Mines)
Dhanbad, India
koustav@iitism.ac.in

Megha Khosla
L3S Research Center
Hannover, Germany
khosla@L3S.de

Abhijit Anand
L3S Research Center
Hannover, Germany
aanand@L3S.de

Avishek Anand
L3S Research Center
Hannover, Germany
anand@L3S.de

ABSTRACT

Neural document ranking approaches, specifically transformer models, have achieved impressive gains in ranking performance. However, query processing using such over-parameterized models is both resource and time intensive. In this paper, we propose the FAST-FORWARD index – a simple vector forward index that facilitates ranking documents using interpolation of lexical and semantic scores – as a replacement for contextual re-rankers and dense indexes based on nearest neighbor search. FAST-FORWARD indexes rely on efficient sparse models for retrieval and merely look up pre-computed dense transformer-based vector representations of documents and passages in constant time for fast CPU-based semantic similarity computation during query processing. We propose index pruning and theoretically grounded early stopping techniques to improve the query processing throughput. We conduct extensive large-scale experiments on TREC-DL datasets and show improvements over hybrid indexes in performance and query processing efficiency using only CPUs. FAST-FORWARD indexes can provide superior ranking performance using interpolation due to the complementary benefits of lexical and semantic similarities.

CCS CONCEPTS

• Information systems → Retrieval models and ranking.

KEYWORDS

retrieval, dense, sparse, ranking, interpolation

ACM Reference Format:

Jurek Leonhardt, Koustav Rudra, Megha Khosla, Abhijit Anand, and Avishek Anand. 2022. Efficient Neural Ranking using Forward Indexes. In *Proceedings of the ACM Web Conference 2022 (WWW '22)*, April 25–29, 2022, Virtual Event, Lyon, France. ACM, New York, NY, USA, 11 pages. <https://doi.org/10.1145/3485447.3511955>

*Research was primarily conducted while affiliated to L3S Research Center.

Permission to make digital or hard copies of all or part of this work for personal or classroom use is granted without fee provided that copies are not made or distributed for profit or commercial advantage and that copies bear this notice and the full citation on the first page. Copyrights for components of this work owned by others than ACM must be honored. Abstracting with credit is permitted. To copy otherwise, or republish, to post on servers or to redistribute to lists, requires prior specific permission and/or a fee. Request permissions from permissions@acm.org.

WWW '22, April 25–29, 2022, Virtual Event, Lyon, France

© 2022 Association for Computing Machinery.

ACM ISBN 978-1-4503-9096-5/22/04...\$15.00

<https://doi.org/10.1145/3485447.3511955>

1 INTRODUCTION

The standard approach for ad-hoc document ranking employs a retrieval phase for fast, high-recall candidate selection and a more expensive re-ranking phase. Recent approaches have focused heavily on neural transformer-based models for both retrieval [6, 18, 24] and re-ranking [1, 4, 12, 14, 22, 29]. However, limited work has been done in the context of efficient end-to-end solutions for ranking long documents.

There are challenges in both retrieval and re-ranking of long documents. The predominant strategy for the retrieval stage is based on *term-based* or *lexical* matching. Towards efficient retrieval, inverted indexes, referred to as *sparse indexes*, have been employed as work horses in traditional information retrieval, exploiting sparsity in the term space for pruning out a large number of irrelevant documents. Sparse indexes are known to suffer from the vocabulary mismatch problem, causing them to rank semantically similar documents low. To alleviate this, *dense indexes* have been proposed recently in the context of passage retrieval. However, we find that recall considerably suffers when retrieving longer documents, i.e. when there are multiple vectors per document. For example, in one case the recall for retrieving 1000 documents using a dense index is 0.58, compared to 0.70 for a sparse index (refer to Table 2).

For the re-ranking stage, cross-attention models are preferred due to their ability to estimate semantic similarity, but these models are computationally expensive. Consequently, to keep overall end-to-end ranking costs manageable, either a smaller number of documents is considered in the re-ranking phase or leaner models with fewer parameters are used, resulting in reduced ranking performance.

The aim of this paper is to propose an efficient end-to-end approach for ranking long documents without compromising effectiveness. Firstly, we observe that dual-encoder models can be used to effectively compute the semantic similarity of a document given a query (cf. Table 1). Based on this observation, we propose a forward indexing approach that ranks documents based on interpolation, scoring them using a linear combination of the semantic similarity scores (from the re-ranking phase) and lexical matching scores (from the retrieval phase) with respect to the query [1].

Our proposed *vector forward index* of documents, referred to as the FAST-FORWARD index, contains a list of pre-computed vectors for each document corresponding to its constituent passages. Query processing using a FAST-FORWARD index entails retrieving documents using a sparse index, computing the semantic similarities

of the retrieved documents through a sequence of index look-ups and dot products and interpolating the scores. As a result, FAST-FORWARD indexes combine the benefits of both sparse and dense models, while at the same time eliminating the need for expensive nearest neighbor search (as used in dense retrieval) or contextual, GPU-accelerated re-ranking. Consequently, our work is complementary to and can be combined with other techniques that aim to improve retrieval, e.g. CLEAR [11] or DocT5QUERY [35]. Further, this allows us to re-rank a much higher number of documents per query, alleviating the aforementioned vocabulary mismatch problem of the sparse retriever.

Our second observation is that query processing cost in the re-ranking phase is dominated by dot-product computations between the query and the passage vectors. Towards this, we propose two techniques to improve efficiency in query processing using FAST-FORWARD indexes – *sequential coalescing* and *early stopping*. In sequential coalescing, we substantially reduce the number of vectors per document by combining representations corresponding to adjacent yet similar passages. This not only improves the query processing efficiency, but also reduces the memory footprint of the index. Early stopping exploits the natural ordering of the sparse scores to avoid unnecessary index look-ups by maintaining a maximum score estimate for the dense scores.

We perform extensive experimental evaluation to show the performance benefits of FAST-FORWARD indexes and our query processing techniques. We find that interpolation using dual-encoder models consistently yields better performance than standard re-ranking using the same models. Further, increasing the sparse retrieval depth prior to interpolation improves the final ranking. Finally, we show how optimized FAST-FORWARD indexes accelerate the ranking, substantially decreasing the query processing time compared to hybrid retrieval, while maintaining comparable performance. To sum up, we make the following contributions:

- We propose FAST-FORWARD indexes as an efficient approach for ad-hoc document ranking tasks.
- We propose novel query processing algorithms – sequential coalescing and early stopping – that further improve the overall query processing throughput.
- We perform extensive experimental evaluation to establish strong efficiency gains due to our forward indexes and query processing techniques.

2 RELATED WORK

Classical ranking approaches, such as BM25 or the query likelihood model [20], rely on the inverted index for efficient first-stage retrieval that stores term-level statistics like term-frequency, inverse document frequency and positional information. We refer to this style of retrieval as sparse retrieval, since it assumes sparse document representations. Recently, Dai and Callan [6, 7] proposed DEEP-CT, which stores contextualized scores for terms in the inverted index for both passage and document retrieval. Similarly, DEEPImpact [31] enriches the document collection with expansion terms to learn improved term impacts. SPLADE [8] aims to enrich sparse document representations using a trained contextual transformer model and sparsity regularization on the term weights. TILDE [43] ranks documents using a deep query and document

likelihood model. In this work, we use the vanilla inverted index with standard term statistics for first-stage retrieval.

An alternative design strategy is to use dual-encoders to learn dense vector representations for passages or documents using contextual models [10, 17, 18]. The dense vectors are then indexed in an offline phase [16], where retrieval is akin to performing an approximate nearest neighbor (ANN) search given a vectorized query. Consequently, there has been a large number of follow-up works that boost the performance of dual-encoder models by improving pre-training [2], optimization techniques [11] and negative sampling [36, 41]. In this work, we use dual-encoders for computing semantic similarity between queries and passages. Some approaches have also proposed architectural modifications to the dual-encoder models by having lightweight aggregations between the query and passage embeddings [3, 12, 15, 25, 26, 41, 42], showing promising performance over standard term-based retrieval strategies. Nogueira et al. [35] proposed a simple document expansion model. Note that these approaches are complementary to our work, as they can be combined with FAST-FORWARD indexes.

Models for Semantic Similarity. While lexical matching models are typically employed in the first-stage retrieval and are known to achieve high recall, the ability to accurately determine semantic similarity is essential in the subsequent more involved and computationally expensive re-ranking stage to alleviate the vocabulary mismatch problem [5, 7, 28, 30, 34]. Computing the semantic similarity of a document given a query has been heavily researched in IR using smoothing methods [19], topic models [40], embeddings [33], personalized models [27] and other techniques. In these classical approaches, ranking is performed by interpolating the semantic similarity scores with the lexical matching scores from the first-stage retrieval. Recent approaches have been dominated by over-parameterized contextual models used predominantly in re-ranking [1, 4, 12, 14, 22, 29]. Unlike dual-encoder models used in dense retrieval, most of the above ranking models take as input a concatenation of the query and document. This combined input results in higher query processing times for large retrieval depths since each document has to be processed in conjunction with the query string. Another key limitation of using contextual models for document ranking is the maximum acceptable number of input tokens for transformer models. Some strategies address this length limitation by document truncation [29] and chunking into passages [4, 37]. However, the performance of chunking-based strategies depends on the chunking properties, i.e. passage length or overlap among consecutive passages [38]. Recent proposals include a two-stage approach, where a query-specific summary is generated by selecting relevant parts of the document, followed by re-ranking strategies over the query and summarized document [13, 23].

Interpolation-based Rankers. Unlike classical methods, where score interpolation is the norm, semantic similarity using a contextual model is not consistently combined with the matching score. Recently, Wang et al. [39] showed that the interpolation of BERT-based retrievers and sparse retrieval methods can boost the performance. Further, they analyzed the role of interpolation in BERT-based dense retrieval strategies (ANCE, RepBERT) and found that dense retrieval alone is not enough, but interpolation with BM25 scores is necessary.

3 INTERPOLATION-BASED RE-RANKING

In this section we formally introduce the problem of re-ranking. We further compare standard and interpolation-based re-ranking.

3.1 Problem Statement

The retrieval of documents or passages given a query typically happens in two stages: In the first stage, a term-frequency-based (**sparse**) retrieval method retrieves a set of documents from a large corpus. In the second stage, another model, which is usually much more computationally expensive, **re-ranks** the retrieved documents again. The re-ranking step is deemed very important for tasks that require high performance for small retrieval depths, such as question answering.

In **sparse retrieval**, we denote the top- k_S documents retrieved from the sparse index for a query q as K_S^q . The sparse score of a query-document pair (q, d) is denoted by $\phi_S(q, d)$. For the **re-ranking** part, we focus on self-attention models (such as BERT) in this work. These models operate by creating (internal) high-dimensional dense representations of queries and documents, focusing on their semantic structure. We refer to the outputs of these models as **dense** or **semantic** scores and denote them by $\phi_D(q, d)$. Due to the quadratic time complexity of self-attention w.r.t. the document length, long documents are often split into passages, and the score of a document is then computed as the maximum of its passage scores:

$$\phi_D(q, d) = \max_{p_i \in d} \phi_D(q, p_i) \quad (1)$$

This approach is referred to as *maxP* [4].

The retrieval approach for a query q starts by retrieving K_S^q from the sparse index. For each retrieved document $d \in K_S^q$, the corresponding dense score $\phi_D(q, d)$ is computed. This dense score may then be used to re-rank the retrieved set to obtain the final ranking. However, recently it has been shown that the scores of the sparse retriever, ϕ_S , can be beneficial for re-ranking as well [1]. To that end, an interpolation approach is employed, where the final score of a query-document pair is computed as follows:

$$\phi(q, d) = \alpha \cdot \phi_S(q, d) + (1 - \alpha) \cdot \phi_D(q, d) \quad (2)$$

Setting $\alpha = 0$ recovers the standard re-ranking procedure.

Since the set of documents retrieved by the sparse model is typically large (e.g. $k_S = 1000$), computing the dense score for each query-document pair can be very computationally expensive. In this paper, we focus on efficient implementations of interpolation-based re-ranking, specifically the computation of the dense scores ϕ_D .

3.2 Re-Ranking vs. Interpolation

In order to compare traditional re-ranking with interpolation (cf. Section 3.1), we retrieve $k_S = 1000$ documents from a sparse BM25 index and compute the corresponding semantic scores for each of the query-document pairs using dense methods. We then re-rank the documents with and without interpolation. The results (cf. Table 1) clearly show that interpolation improves over the purely semantic scoring in both document and passage re-ranking. This

	Doc'19	Doc'20	PASSAGE'19
RE-RANKING			
TCT-COLBERT	0.685	0.617	0.694
BERT-CLS	0.520	0.522	0.578
INTERPOLATION			
TCT-COLBERT	0.696	0.637	0.708
BERT-CLS	0.612	0.626	0.617

Table 1: Performance comparison (nDCG@10) between re-ranking and interpolation at retrieval depth $k_S = 1000$.

confirms that the lexical scores indeed hold complementary relevance signals in comparison to semantic scores. We also observe that the dual-encoder approach has better re-ranking performance.

4 EFFICIENT INTERPOLATION

Interpolation-based re-ranking, as described in Section 3, is known to improve performance when applied to the results of a first-stage (sparse) retrieval step. However, the computation of ϕ_D (cf. Equation (2)) can be very slow, where cross-attention models [4, 37] are more expensive than dual-encoder-based ranking strategies [2, 26]. In this section we propose several means of implementing interpolation-based re-ranking more efficiently.

4.1 Hybrid Retrieval

Hybrid retrieval is similar to standard interpolation-based re-ranking (cf. Section 3). The key difference is that the dense scores $\phi_D(q, d)$ are not computed for all query-document pairs. Instead, this approach operates under the assumption that ϕ_D is a **dense retrieval model**, which retrieves documents d_i and their scores $\phi(q, d_i)$ using nearest neighbor search given a query q . A hybrid retriever combines the retrieved sets of a sparse and a dense retriever.

For a query q , we retrieve two sets of documents, K_S^q and K_D^q , using the sparse and dense retriever, respectively. Note that the two retrieved sets are usually not equal. One strategy proposed in [26] ranks all documents in $K_S^q \cup K_D^q$, approximating missing scores. In our experiments, however, we found that **only** considering documents from K_S^q for the final ranking and discarding the rest works well. The final score is thus computed as follows:

$$\phi(q, d) = \alpha \cdot \phi_S(q, d) + (1 - \alpha) \cdot \begin{cases} \phi_D(q, d) & d \in K_D^q \\ \phi_S(q, d) & d \notin K_D^q \end{cases} \quad (3)$$

The re-ranking step in hybrid retrieval is essentially a sorting operation over the interpolated scores and takes negligible time in comparison to standard re-ranking.

4.2 FAST-FORWARD INDEXES

The hybrid approach described in Section 4.1 has two distinct disadvantages. Firstly, in order to retrieve K_D^q , an (approximate) nearest neighbor search has to be performed, which is time consuming. Secondly, some of the query-document scores are missed, leading to an incomplete interpolation.

In this section we propose FAST-FORWARD indexes as an efficient way of computing dense scores for known documents that alleviates

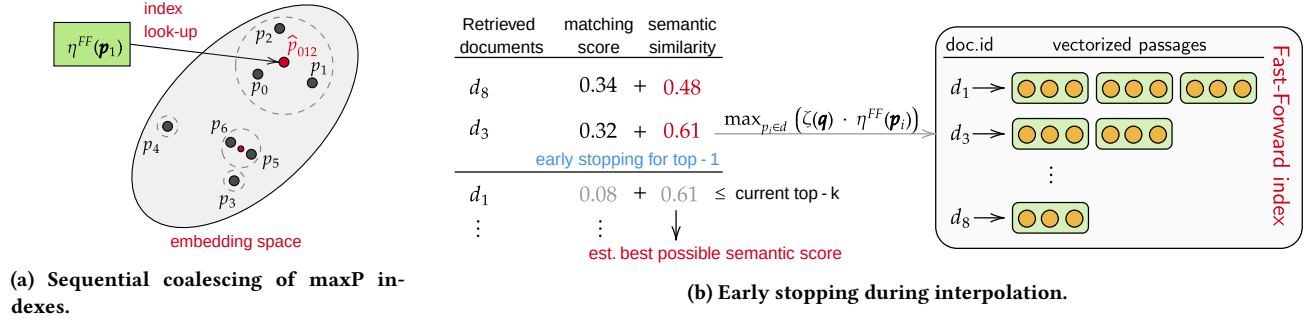


Figure 1: The optimization techniques that can be applied to FAST-FORWARD indexes. Sequential coalescing combines the representations of similar consecutive passages as their average. Note that p_3 and p_5 are not combined, as they are not consecutive passages. Early stopping reduces the number of interpolation steps by computing an approximate upper bound for the dense scores. This example depicts the most extreme case, where only the top-1 document is required.

the aforementioned issues. Specifically, FAST-FORWARD indexes build upon two-tower dense retrieval models that compute the score of a query-document pair as a dot product

$$\phi_D(q, d) = \zeta(q) \cdot \eta(d), \quad (4)$$

where ζ and η are the query and document encoders, respectively. Examples of such models are ANCE [41] and TCT-ColBERT [26]. Since the query and document representations are independent for two-tower models, we can pre-compute the document representations $\eta(d)$ for each document d in the corpus. These document representations are then stored in an efficient hash map, allowing for look-ups in constant time. After the index is created, the score of a query-document pair can be computed as

$$\phi_D^{FF}(q, d) = \zeta(q) \cdot \eta^{FF}(d), \quad (5)$$

where the superscript FF indicates the look-up of a pre-computed document representation in the FAST-FORWARD index. At retrieval time, only $\zeta(q)$ needs to be computed once for each query. As queries are usually short, this can be done on CPUs.

4.3 Index Compression via Seq. Coalescing

A major disadvantage of dense indexes and dense retrieval in general is the size of the final index. This is caused by two factors: Firstly, in contrast to sparse indexes, the high-dimensional dense representations can not be stored as efficiently as sparse vectors. Secondly, the dense encoders are typically transformer-based, imposing a (soft) limit on their input lengths due to their quadratic time complexity with respect to the inputs. Thus, long documents are split into passages prior to indexing (*maxP* indexes).

As an increase in the index size has a negative effect on retrieval latency, both for nearest neighbor search and FAST-FORWARD indexing as used by our approach, we exploit a *sequential coalescing* approach as a way of dynamically combining the representations of consecutive passages within a single document in maxP indexes. The idea is to reduce the number of passage representations in the index for a single document. This is achieved by exploiting the *topical locality* that is inherent to documents [21]. For example, a single document might contain information regarding multiple topics; due to the way human readers naturally ingest information, we expect documents to be authored such that a single topic appears

Algorithm 1: Compression of dense maxP indexes by sequential coalescing

Input: list of passage vectors P (original order) of a document, distance threshold δ

Output: coalesced passage vectors P'

```

1  $P' \leftarrow$  empty list
2  $\mathcal{A} \leftarrow \emptyset$ 
3 foreach  $v$  in  $P$  do
4   if first iteration then
5     // do nothing
6   else if  $\text{cosine\_distance}(v, \overline{\mathcal{A}}) \geq \delta$  then
7     append  $\overline{\mathcal{A}}$  to  $P'$ 
8      $\mathcal{A} \leftarrow \emptyset$ 
9     add  $v$  to  $\mathcal{A}$ 
10     $\overline{\mathcal{A}} \leftarrow \text{mean}(\mathcal{A})$ 
11 end
12 append  $\overline{\mathcal{A}}$  to  $P'$ 
13 return  $P'$ 

```

mostly in consecutive passages, rather than spread throughout the whole document. Our approach aims to combine consecutive passage representations that encode similar information. To that end, we employ the cosine distance function and a *threshold* parameter δ that controls the degree of coalescing. Within a single document, we iterate over its passage vectors in their original order and maintain a set $\mathcal{A}$, which contains the representations of the already processed passages, and continuously compute $\overline{\mathcal{A}}$ as the average of all vectors in $\mathcal{A}$. For each new passage vector v , we compute its cosine distance to $\overline{\mathcal{A}}$. If it exceeds the distance threshold δ , the current passages in $\mathcal{A}$ are combined as their average representation $\overline{\mathcal{A}}$. Afterwards, the combined passages are removed from $\mathcal{A}$ and $\overline{\mathcal{A}}$ is recomputed. This approach is illustrated in Algorithm 1. Figure 1a shows an example index after coalescing. To the best of our knowledge, there are no other forward index compression techniques so far.

Algorithm 2: Interpolation with early stopping

Input: query q , sparse retrieval depth k_S , cut-off depth k , interpolation parameter α
Output: approximated top- k scores Q

```

1  $Q \leftarrow$  priority queue of size  $k$ 
2  $s_D \leftarrow -\infty$ 
3  $s_{min} \leftarrow -\infty$ 
4 foreach  $d$  in  $\text{sparse}(q, k_S)$  do
5   if  $Q$  is full then
6      $s_{min} \leftarrow$  remove smallest item from  $Q$ 
7      $s_{best} \leftarrow \alpha \cdot \phi_S(q, d) + (1 - \alpha) \cdot s_D$ 
8     if  $s_{best} \leq s_{min}$  then
9       // early stopping
10      put  $s_{min}$  into  $Q$ 
11      break
12   // approximate max. dense score
13    $s_D \leftarrow \max(\phi_D(q, d), s_D)$ 
14    $s \leftarrow \alpha \cdot \phi_S(q, d) + (1 - \alpha) \cdot \phi_D(q, d)$ 
15   put  $\max(s, s_{min})$  into  $Q$ 
16 end
17 return  $Q$ 

```

4.4 Faster Interpolation by Early Stopping

As described in Section 3, by interpolating the scores of sparse and dense retrieval models we perform implicit re-ranking, where the dense representations are pre-computed and can be looked up in a FAST-FORWARD index at retrieval time. Further, increasing the sparse retrieval depth k_S , such that $k_S > k$, where k is the final number of documents, improves the performance. A drawback of this is that an increase in the number of retrieved documents also results in an increase in the number of index look-ups.

In this section we propose an extension to FAST-FORWARD indexes that allows for *early stopping*, i.e. avoiding a number of unnecessary look-ups, for cases where $k_S > k$ by approximating the maximum possible dense score. The early stopping approach takes advantage of the fact that documents are ordered by their sparse scores $\phi_S(q, d)$. Since the number of retrieved documents, k_S , is finite, there exists an upper limit s_D for the corresponding dense scores such that $\phi_D(q, d) \leq s_D \forall d \in K_S^q$. Since the retrieved documents K_S^q are ordered by their sparse scores, we can simultaneously perform interpolation and re-ranking by iterating over the ordered list of documents: Let d_i be the i -th highest ranked document by the sparse retriever. Recall that we compute the final score as follows:

$$\phi(q, d_i) = \alpha \cdot \phi_S(q, d_i) + (1 - \alpha) \cdot \phi_D(q, d_i) \quad (6)$$

If $i > k$, we can compute the upper bound for $\phi(q, d_i)$ by exploiting the aforementioned ordering:

$$s_{best} = \alpha \cdot \phi_S(q, d_{i-1}) + (1 - \alpha) \cdot s_D \quad (7)$$

In turn, this allows us to stop the interpolation and re-ranking if $s_{best} \leq s_{min}$, where s_{min} denotes the score of the k -th document in the current ranking (i.e. the currently lowest ranked document). Intuitively, this means that we stop the computation once the *highest possible* interpolated score $\phi(q, d_i)$ is too low to make a difference.

The approach is illustrated in Algorithm 2 and Figure 1b. Since the dense scores ϕ_D are usually unnormalized, the upper limit s_D is unknown in practice. We thus approximate it by using the highest observed dense score at any given step.

4.4.1 Theoretical Analysis. We first show that the early stopping criteria, when using the true maximum of the dense scores, is sufficient to obtain the top- k scores.

THEOREM 4.1. *Let s_D , as used in Algorithm 2, be the true maximum of the dense scores. Then the returned scores are the actual top- k scores.*

PROOF. First, note that the sparse scores, $\phi_S(q, d_i)$, are already sorted in decreasing order for a given query. By construction, the priority queue Q always contains the highest scores corresponding to the list parsed so far. Let, after parsing k scores, Q be full. Now the possible best score s_{best} is computed using the sparse score found next in the decreasing sequence and the maximum of all dense scores, s_D (cf. line 7). If s_{best} is less than the minimum of the scores in Q , then Q already contains the top- k scores. To see this, note that the first component of s_{best} is the largest among all unseen sparse scores (as the list is sorted) and s_D is maximum of the dense scores by our assumption. $\square$

Next, we show that a good approximation of the top- k scores can be achieved by using the sample maximum. To prove our claim, we use the Dvoretzky–Kiefer–Wolfowitz (DKW) [32] inequality.

LEMMA 4.2. *Let $X_1, X_2, \dots, X_n$ be n real-valued independent and identically distributed random variables with the cumulative distribution function $F(\cdot)$. Let $F_n(\cdot)$ denote the empirical cumulative distribution function, i.e.*

$$F_n(x) = \frac{1}{n} \sum_{i=1}^n \mathbb{1}_{\{X_i \leq x\}}, \quad x \in \mathbb{R}. \quad (8)$$

According to the DKW inequality, the following estimate holds:

$$\Pr \left(\sup_{x \in \mathbb{R}} (F_n(x) - F(x)) > \epsilon \right) \leq e^{-2n\epsilon^2} \forall \epsilon \geq \sqrt{\frac{1}{2n} \ln 2}. \quad (9)$$

In the following we show that, if s_D is chosen as the maximum of a large random sample drawn from the set of dense scores, then the probability that any given dense score, chosen independently and uniformly at random from the dense scores, is greater than s_D is exponentially small in the sample size.

THEOREM 4.3. *Let $x_1, x_2, \dots, x_n$ be a real-valued independent and identically distributed random sample drawn from the distribution of the dense scores with the cumulative distribution function $F(\cdot)$. Let $z = \max(x_1, x_2, \dots, x_n)$. Then, for every $\epsilon > \frac{1}{\sqrt{2n}} \ln 2$, we obtain*

$$\Pr(F(z) < 1 - \epsilon) \leq e^{-2n\epsilon^2}. \quad (10)$$

PROOF. Let $F_n(\cdot)$ denote the empirical cumulative distribution function as above. Specifically, $F_n(x)$ is equal to the fraction of variables less than or equal to x . We then have $F_n(z) = 1$. By Lemma 4.2, we infer

$$\Pr(F_n(z) - F(z) > \epsilon) \leq e^{-2n\epsilon^2}. \quad (11)$$

Substituting $F_n(z) = 1$, we obtain Equation (10). $\square$

This implies that the probability of any random variable X , chosen randomly from the set of dense scores, being less than or equal to s_D , is greater than or equal to $1 - \epsilon$ with high probability, i.e.

$$\Pr(P_D(X \leq s_D) \geq 1 - \epsilon) \geq 1 - e^{-2n\epsilon^2}, \quad (12)$$

where P_D denotes the probability distribution of the dense scores. This means that, as our sample size grows until it reaches k , the approximation improves. Note that, in our case, the dense scores are sorted (by corresponding sparse score) and thus the i.i.d. assumption can not be ensured. However, we observed that the dense scores are positively correlated with the sparse scores. We argue that, due to this correlation, we can approximate the maximum score well.

5 EVALUATION SETUP

We consider the following baselines:

Lexical or sparse retrievers rely on term-based matching between queries and documents. We consider BM25, which uses term-based retrieval signals, and DEEP-CT [6], which is similar to BM25, but the term weights are learned in a contextualized fashion.

Semantic or dense retrievers retrieve documents that are semantically similar to the query in a common embedding space. We consider TCT-COLBERT [26] and ANCE [41]. Both approaches are based on BERT encoders. Large documents are split into passages before indexing (maxP). These dense retrievers use exact (brute-force) nearest neighbor search as opposed to approximate nearest neighbor (ANN) search.

Hybrid retrievers interpolate sparse and dense retriever scores. We consider CLEAR [11], a retrieval model that complements lexical models with semantic matching. Additionally, we consider the hybrid strategy described in Section 4.1 as a baseline, using the dense retrievers above.

Contextual dense re-rankers operate on the documents retrieved by a sparse retriever (BM25). Each query-document pair is input into the re-ranker, which outputs a corresponding score. In this paper, we use a BERT-CLS re-ranker, where the output corresponding to the classification token is used as the score. Note that re-ranking is performed using the full documents (i.e. documents are not split into passages). If an input exceeds 512 tokens, it is truncated.

Datasets and Hyperparameters. We conduct experiments on three datasets from the TREC Deep Learning track, Doc'19, Doc'20 and PASSAGE'19, to evaluate the effectiveness and efficiency of retrieval and re-ranking strategies on the MS MARCO collection. Each test set has a total of 200 queries. We use the PYSERINI toolkit [25] for our retrieval experiments and the MS MARCO development set to determine $\alpha = 0.2$ for TCT-COLBERT, $\alpha = 0.5$ for ANCE and $\alpha = 0.7$ for BERT-CLS. Latency is computed as the sum of scoring, interpolation and sorting cost. Tokenization cost is ignored. We report the average processing time per query in the test set. Where applicable, dense models use a batch size of 256. More details can be found in Appendix A.1.

6 EXPERIMENTAL RESULTS

In this section we perform large-scale experiments to show the effectiveness and efficiency of the proposed FAST-FORWARD indexes.

RQ1. How does interpolation-based re-ranking with dual-encoders compare to other methods? In Table 2, we report the performance of sparse, dense and hybrid retrievers, re-rankers and interpolation.

First, we observe that dense retrieval strategies perform better than sparse ones in terms of nDCG, but have poor recall except on PASSAGE'19. The contextual weights learned by DEEP-CT are better than tf-idf based retrieval (BM25), but fall short of dense semantic retrieval strategies (TCT-COLBERT and ANCE). However, the overlap among retrieved documents is rather low, reflecting that dense retrieval cannot match query and document terms well.

Second, dual-encoder-based (TCT-COLBERT and ANCE) perform better than contextual (BERT-CLS) re-rankers. In this setup, we first retrieve $k_S = 1000$ documents using a sparse retriever and re-rank them. This approach benefits from high recall in the first stage and promotes the relevant documents to the top of the list through the dense semantic re-ranker. However, re-ranking is typically time-consuming and requires GPU acceleration. The improvement of TCT-COLBERT and ANCE over BERT-CLS also suggests that dual-encoder-based re-ranking strategies are better than cross-interaction-based methods. However, the difference could also be attributed to the fact that BERT-CLS does not follow the maxP approach (cf. Section 3.1).

Finally, interpolation-based re-ranking, which combines the benefits of sparse and dense scores, significantly outperforms the BERT-CLS re-ranker and dense retrievers. Recall that dense re-rankers operate solely based on the dense scores and discard the sparse BM25 scores of the query-document pairs. The superiority of interpolation-based methods is also supported by evidence from recent studies [2, 3, 10, 11].

RQ2. Do FAST-FORWARD indexes allow for efficient interpolation at higher retrieval depths? Tables 3 and 4 show results of re-ranking, hybrid retrieval and interpolation on document and passage datasets, respectively. The metrics are computed for two sparse retrieval depths, $k_S = 1000$ and $k_S = 5000$.

We observe that taking the sparse component into account in the score computation (as is done by the interpolation and hybrid methods) causes performance to improve with retrieval depth. Specifically, some queries receive a considerable recall boost, capturing more relevant documents with large retrieval depths. Interpolation based on FAST-FORWARD indexes achieves substantially lower latency compared to other methods. Pre-computing the document representations allows for fast look-ups during retrieval time. As only the query needs to be encoded by the dense model, both retrieval and re-ranking can be performed on the CPU while still offering considerable improvements in query processing time. Note that for BERT-CLS, the input length is limited, causing documents to be truncated, similarly to the *firstP* approach. As a result, the latency is much lower, but in turn the performance suffers. It is important to note here, that, in principle, FAST-FORWARD indexes can also be used in combination with *firstP* models.

The hybrid retrieval strategy, as described in Section 4.1, shows good performance. However, as the dense indexes require nearest neighbor search for retrieval, the query processing latency is much higher than for interpolation using FAST-FORWARD indexes.

Finally, dense re-rankers do not profit reliably from increased sparse retrieval depth; on the contrary, the performance drops

	Doc'19			Doc'20			PASSAGE'19		
	AP _{1k}	R _{1k}	nDCG ₁₀	AP _{1k}	R _{1k}	nDCG ₁₀	AP _{1k}	R _{1k}	nDCG ₁₀
SPARSE RETRIEVAL									
BM25	0.331	0.697	0.519 ¹⁻³	0.404	0.809	0.527 ¹⁻³	0.301	0.750	0.506 ¹⁻³
DEEP-CT	-	-	0.544	-	-	-	0.422	0.756	0.551
DENSE RETRIEVAL									
TCT-ColBERT	0.279	0.576	0.612 ¹	0.372	0.728	0.586 ^{1,2}	0.391	0.792	0.670
ANCE	0.254	0.510	0.633 ¹	0.401	0.681	0.633	0.371	0.755	0.645
HYBRID RETRIEVAL									
CLEAR	-	-	-	-	-	-	0.511	0.812	0.699
RE-RANKING									
TCT-ColBERT	0.370	0.697	0.685	0.414	0.809	0.617	0.423	0.750	0.694
ANCE	0.336	0.697	0.654	0.426	0.809	0.630	0.389	0.750	0.679
BERT-CLS	0.283	0.697	0.520 ¹⁻³	0.329	0.809	0.522 ¹⁻³	0.353	0.750	0.578 ^{1,2}
INTERPOLATION									
TCT-ColBERT ¹	0.406	0.697	0.696	0.469	0.809	0.637	0.438	0.750	0.708
ANCE ²	0.387	0.697	0.673	0.490	0.809	0.655	0.417	0.750	0.680
BERT-CLS ³	0.365	0.697	0.612	0.460	0.809	0.626	0.378	0.750	0.617

Table 2: Retrieval performance. Retrievers use depths $k_S = 1000$ (sparse) and $k_D = 10000$ (dense). Dense retrievers retrieve passages and perform maxP aggregation for documents. Scores for CLEAR and DEEP-CT are taken from the corresponding papers [10, 11]. Superscripts indicate statistically significant improvements using two-paired tests with a sig. level of 95% [9].

	millisec. per query	Doc'19						Doc'20					
		$k_S = 1000$			$k_S = 5000$			$k_S = 1000$			$k_S = 5000$		
		AP _{1k}	R _{1k}	nDCG ₂₀	AP _{1k}	R _{1k}	nDCG ₂₀	AP _{1k}	R _{1k}	nDCG ₂₀	AP _{1k}	R _{1k}	nDCG ₂₀
HYBRID RETRIEVAL													
BM25, TCT-ColBERT	582	0.394	0.697	0.655	0.385	0.729	0.645	0.463	0.809	0.615	0.469	0.852	0.621
BM25, ANCE	582	0.379	0.697	0.633	0.373	0.727	0.628	0.479	0.809	0.624	0.488	0.846	0.632
RE-RANKING													
TCT-ColBERT	1189 + 2	0.370	0.697	0.632	0.334	0.703	0.609 ¹	0.414	0.809	0.587 ¹	0.405	0.794	0.585 ^{1,3,4}
ANCE	1189 + 2	0.336	0.697	0.614	0.304	0.647	0.607	0.426	0.809	0.595 ³	0.422	0.761	0.604
BERT-CLS	185 + 2	0.283	0.697	0.494 ¹⁻⁵	0.159	0.559	0.289	0.329	0.809	0.512 ¹⁻⁵	0.221	0.727	0.375 ¹⁻⁵
INTERPOLATION													
TCT-ColBERT ¹	1189 + 14	0.406	0.697	0.655	0.411	0.745	0.653	0.469	0.809	0.621	0.478	0.838	0.626
FAST-FORWARD	253	0.406	0.697	0.655	0.411	0.745	0.653	0.469	0.809	0.621	0.478	0.838	0.626
coalesced ²	109	0.379	0.697	0.630	0.379	0.732	0.625	0.440	0.809	0.594 ¹	0.447	0.837	0.607
ANCE ³	1189 + 14	0.387	0.697	0.638	0.393	0.732	0.639	0.490	0.809	0.630	0.502	0.828	0.640
FAST-FORWARD	253	0.387	0.697	0.638	0.393	0.732	0.639	0.490	0.809	0.630	0.502	0.828	0.640
coalesced ⁴	121	0.372	0.697	0.625	0.375	0.723	0.628	0.471	0.809	0.622	0.479	0.823	0.629
BERT-CLS ⁵	185 + 14	0.365	0.697	0.585	0.357	0.708	0.562	0.460	0.809	0.602	0.459	0.839	0.601

Table 3: Document retrieval performance. Latency is reported for $k_S = 5000$ on CPU and GPU. The coalesced FAST-FORWARD indexes are compressed to approximately 25% of their original size. Hybrid retrievers use a dense retrieval depth of $k_D = 1000$. Superscripts indicate statistically significant improvements using two-paired tests with a sig. level of 95% [9].

in some cases. This trend is more apparent for the document retrieval datasets with higher values of k_S . We hypothesize that dense rankers only focus on semantic matching and are sensitive to topic drift, causing them to rank irrelevant documents in the top-5000 higher.

RQ3. Can the re-ranking efficiency be improved by reducing the FAST-FORWARD index size using sequential coalescing? In order to evaluate this approach, we first take the pre-trained TCT-ColBERT dense index of the MS MARCO corpus, apply sequential coalescing with varying values for δ and evaluate each resulting compressed

	millisec. per query	$k_S = 1000$		$k_S = 5000$	
		AP _{1k}	RR ₁₀	AP _{1k}	RR ₁₀
HYBRID RETRIEVAL					
BM25, TCT-ColBERT	307	0.434	0.894	0.454	0.902
BM25, ANCE	307	0.410	0.856	0.422	0.864
RE-RANKING					
TCT-ColBERT	186 + 2	0.426	0.827	0.439	0.842
ANCE	186 + 2	0.389	0.836	0.392	0.857
BERT-CLS	185 + 2	0.353	0.715	0.275	0.576
INTERPOLATION					
TCT-ColBERT	186 + 14	0.438	0.894	0.460	0.902
FAST-FORWARD	114	0.438	0.894	0.460	0.902
early stopping	72	-	0.894	-	0.902
ANCE	186 + 14	0.417	0.856	0.435	0.864
FAST-FORWARD	114	0.417	0.856	0.435	0.864
early stopping	52	-	0.856	-	0.864
BERT-CLS	185 + 14	0.378	0.809	0.392	0.832

Table 4: Retrieval performance on PASSAGE'19. Latency is reported for $k_S = 5000$ on CPU and GPU. Hybrid retrievers use a dense retrieval depth of $k_D = 1000$.

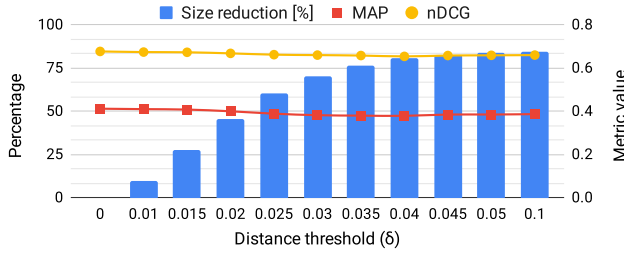


Figure 2: Sequential coalescing applied to Doc'19. The plot shows the index size reduction in terms of the number of passages and the corresponding metric values for FAST-FORWARD interpolation with TCT-ColBERT.

index using the Doc'19 testset. The results are illustrated in Figure 2. It is evident that, by combining the passage representations, the number of vectors in the index can be reduced by more than 80% in the most extreme case, where only a single vector per document remains. At the same time, the performance is correlated with the granularity of the representations. However, the drops are relatively small. For example, for $\delta = 0.025$, the index size is reduced by more than half, while the nDCG decreases by roughly 0.015 (3%).

Additionally, Table 3 shows the detailed performance of coalesced FAST-FORWARD indexes on the document datasets. We chose the indexes corresponding to $\delta = 0.035$ (TCT-ColBERT) and $\delta = 0.003$ (ANCE), both of which are compressed to approximately 25% of their original size. This is reflected in the query processing latency, which is reduced by more than half. The overall performance drops to some extent, as expected, however, these drops are not statistically significant in all but one case. The trade-off between latency (index size) and performance can be controlled by varying the threshold δ .

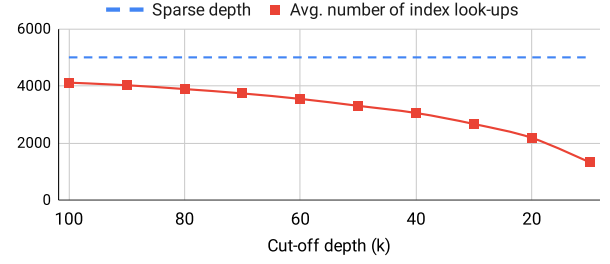


Figure 3: The average number of FAST-FORWARD index look-ups per query for interpolation with early stopping at varying cut-off depths k on PASSAGE'19 with $k_S = 5000$ using ANCE.

RQ4. Can the re-ranking efficiency be improved by limiting the number of FAST-FORWARD look-ups? We start by evaluating the utility of the early stopping approach described in Section 4.4 on the PASSAGE'19 dataset. Figure 3 shows the average number of look-ups performed in the FAST-FORWARD index during interpolation w.r.t. the cut-off depth k . We observe that, for $k = 100$, early stopping already leads to a reduction of almost 20% in the number of look-ups. Decreasing k further leads to a significant reduction of look-ups, resulting in improved query processing latency. As lower cut-off depths (i.e. $k < 100$) are typically used in downstream tasks, such as question answering, the early stopping approach for low values of k turns out to be particularly helpful.

Table 4 shows early stopping applied to the passage dataset to retrieve the top-10 passages and compute reciprocal rank. It is evident that, even though the algorithm approximates the maximum dense score (cf. Section 4.4), the resulting performance is identical, which means that the approximation was accurate in both cases and did not incur any performance hit. Further, the query processing time is decreased by up to a half compared to standard interpolation. Note that early stopping depends on the value of α , hence the latency varies between TCT-ColBERT and ANCE.

7 CONCLUSION

In this paper we propose FAST-FORWARD indexes, a simple yet effective and efficient look-up-based interpolation method that combines document retrieval and re-ranking. FAST-FORWARD indexes are based on dense dual-encoder models, exploiting the fact that document representations can be pre-processed and stored, providing efficient access in constant time. Using interpolation, we observe increased performance compared to hybrid retrieval. Further, we achieve improvements of up to 75% in memory footprint and query processing latency due to our optimization techniques, *sequential coalescing* and *early stopping*. At the same time, our method solely requires CPU computations, completely eliminating the need for expensive GPU-accelerated re-ranking.

ACKNOWLEDGMENTS

Funding for this project was in part provided by EU Horizon 2020 grant no. 871042 (*SoBigData++*) and 832921 (*MIRROR*) and BMBF grant no. 01DD20003 (*LeibnizKILabor*).

REFERENCES

- [1] Zeynep Akkalyoncu Yilmaz, Wei Yang, Haotian Zhang, and Jimmy Lin. 2019. Cross-Domain Modeling of Sentence-Level Evidence for Document Retrieval. In *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing and the 9th International Joint Conference on Natural Language Processing (EMNLP-IJCNLP)*. 3481–3487.
- [2] Wei-Cheng Chang, Felix X. Yu, Yin-Wen Chang, Yiming Yang, and Sanjiv Kumar. 2020. Pre-training Tasks for Embedding-based Large-scale Retrieval. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=rkg-mA4FDr>
- [3] Xiaoyang Chen, Kai Hui, Ben He, Xianpei Han, Le Sun, and Zheng Ye. 2021. Co-BERT: A Context-Aware BERT Retrieval Model Incorporating Local and Query-specific Context. *arXiv preprint arXiv:2104.08523* (2021).
- [4] Zhuyun Dai and Jamie Callan. 2019. Deeper Text Understanding for IR with Contextual Neural Language Modeling. In *ACM SIGIR'19*. 985–988.
- [5] Zhuyun Dai and Jamie Callan. 2019. An Evaluation of Weakly-Supervised DeepCT in the TREC 2019 Deep Learning Track. In *TREC*.
- [6] Zhuyun Dai and Jamie Callan. 2020. Context-Aware Document Term Weighting for Ad-Hoc Search. In *Proceedings of The Web Conference 2020*. 1897–1907.
- [7] Zhuyun Dai and Jamie Callan. 2020. Context-Aware Term Weighting For First Stage Passage Retrieval. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1533–1536.
- [8] Thibault Formal, Benjamin Piwowarski, and Stéphane Clinchant. 2021. SPLADE: Sparse Lexical and Expansion Model for First Stage Ranking. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '21)*. Association for Computing Machinery, New York, NY, USA, 2288–2292. <https://doi.org/10.1145/3404835.3463098>
- [9] Luke Gallagher. 2019. Pairwise t-test on TREC Run Files. <https://github.com/lgrz/pairwise-ttest/>.
- [10] Luyu Gao, Zhuyun Dai, and Jamie Callan. 2021. COIL: Revisit Exact Lexical Match in Information Retrieval with Contextualized Inverted List. In *Proceedings of the 2021 Conference of the North American Chapter of the Association for Computational Linguistics: Human Language Technologies*. Association for Computational Linguistics, Online, 3030–3042. <https://doi.org/10.18653/v1/2021.naacl-main.241>
- [11] Luyu Gao, Zhuyun Dai, Tongfei Chen, Zhen Fan, Benjamin Van Durme, and Jamie Callan. 2021. Complement Lexical Retrieval Model with Semantic Residual Embeddings. In *Advances in Information Retrieval*, Djoerd Hiemstra, Marie-Francine Moens, Josiane Mothe, Raffaele Perego, Martin Potthast, and Fabrizio Sebastiani (Eds.). Springer International Publishing, Cham, 146–160.
- [12] Sebastian Hofstätter, Sheng-Chieh Lin, Jheng-Hong Yang, Jimmy Lin, and Allan Hanbury. 2021. Efficiently Teaching an Effective Dense Retriever with Balanced Topic Aware Sampling. Association for Computing Machinery, New York, NY, USA, 113–122. <https://doi.org/10.1145/3404835.3462891>
- [13] Sebastian Hofstätter, Bhaskar Mitra, Hamed Zamani, Nick Craswell, and Allan Hanbury. 2021. Intra-Document Cascading: Learning to Select Passages for Neural Document Ranking. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1349–1358.
- [14] Sebastian Hofstätter, Markus Zlabinger, and Allan Hanbury. 2020. Interpretable & Time-Budget-Constrained Contextualization for Re-Ranking. In *Proc. of ECAI*.
- [15] Kyoung-Rok Jang, Junmo Kang, Giwon Hong, Sung-Hyon Myaeng, Joohee Park, Taewon Yoon, and Heecheol Seo. 2021. Ultra-High Dimensional Sparse Representations with Binarization for Efficient Text Retrieval. In *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing*. Association for Computational Linguistics, Online and Punta Cana, Dominican Republic, 1016–1029. <https://doi.org/10.18653/v1/2021.emnlp-main.78>
- [16] Jeff Johnson, Matthijs Douze, and Hervé Jégou. 2019. Billion-scale similarity search with gpus. *IEEE Transactions on Big Data* (2019).
- [17] Vladimir Karpukhin, Barlas Oguz, Sewon Min, Patrick Lewis, Ledell Wu, Sergey Edunov, Danqi Chen, and Wen-tau Yih. 2020. Dense Passage Retrieval for Open-Domain Question Answering. In *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, Online, 6769–6781. <https://doi.org/10.18653/v1/2020.emnlp-main.550>
- [18] Omar Khattab and Matei Zaharia. 2020. ColBERT: Efficient and Effective Passage Search via Contextualized Late Interaction over BERT. Association for Computing Machinery, New York, NY, USA, 39–48. <https://doi.org/10.1145/3397271.3401075>
- [19] John Lafferty and Chengxiang Zhai. 2001. Document Language Models, Query Models, and Risk Minimization for Information Retrieval. In *Proceedings of the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '01)*. Association for Computing Machinery, New York, NY, USA, 111–119. <https://doi.org/10.1145/383952.383970>
- [20] Victor Lavrenko and W. Bruce Croft. 2001. Relevance Based Language Models. In *Proceedings of the 24th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '01)*. ACM, New York, NY, USA, 120–127. <https://doi.org/10.1145/383952.383972>
- [21] Jurek Leonhardt, Avishek Anand, and Megha Khosla. 2020. Boilerplate Removal Using a Neural Sequence Labeling Model. In *Companion Proceedings of the Web Conference 2020 (WWW '20)*. Association for Computing Machinery, New York, NY, USA, 226–229. <https://doi.org/10.1145/3366424.3383547>
- [22] Canjia Li, Andrew Yates, Sean MacAvaney, Ben He, and Yingfei Sun. 2020. PA-RADE: Passage Representation Aggregation for Document Reranking. *arXiv preprint arXiv:2008.09093* (2020).
- [23] Minghan Li and Eric Gaussier. 2021. KeyBLD: Selecting Key Blocks with Local Pre-Ranking for Long Document Information Retrieval. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 2207–2211. <https://doi.org/10.1145/3404835.3463083>
- [24] Jimmy Lin. 2019. The Neural Hype and Comparisons Against Weak Baselines. In *ACM SIGIR Forum*, Vol. 52. ACM, 40–51.
- [25] Jimmy Lin, Xueguang Ma, Sheng-Chieh Lin, Jheng-Hong Yang, Ronak Pradeep, and Rodrigo Nogueira. 2021. Pyserini: A Python Toolkit for Reproducible Information Retrieval Research with Sparse and Dense Representations. Association for Computing Machinery, New York, NY, USA, 2356–2362. <https://doi.org/10.1145/3404835.3463238>
- [26] Sheng-Chieh Lin, Jheng-Hong Yang, and Jimmy Lin. 2021. In-Batch Negatives for Knowledge Distillation with Tightly-Coupled Teachers for Dense Retrieval. In *Proceedings of the 6th Workshop on Representation Learning for NLP (ReplANLP-2021)*. Association for Computational Linguistics, Online, 163–173. <https://doi.org/10.18653/v1/2021.repl4nlp-1.17>
- [27] Julia Luxemburger, Shady Elbassouni, and Gerhard Weikum. 2008. Matching Task Profiles and User Needs in Personalized Web Search. In *Proceedings of the 17th ACM Conference on Information and Knowledge Management (CIKM '08)*. Association for Computing Machinery, New York, NY, USA, 689–698. <https://doi.org/10.1145/1458082.1458175>
- [28] Sean MacAvaney, Franco Maria Nardini, Raffaele Perego, Nicola Tonellotto, Nazli Goharian, and Ophir Frieder. 2020. Expansion via Prediction of Importance with Contextualization. Association for Computing Machinery, New York, NY, USA, 1573–1576. <https://doi.org/10.1145/3397271.3401262>
- [29] Sean MacAvaney, Andrew Yates, Arman Cohan, and Nazli Goharian. 2019. CEDR: Contextualized Embeddings for Document Ranking. In *Proceedings of the 42nd International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR'19)*. Association for Computing Machinery, New York, NY, USA, 1101–1104. <https://doi.org/10.1145/3331184.3331317>
- [30] Joel Mackenzie, Zhuyun Dai, Luke Gallagher, and Jamie Callan. 2020. Efficiency Implications of Term Weighting for Passage Retrieval. In *Proceedings of the 43rd International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1821–1824.
- [31] Antonio Mallia, Omar Khattab, Torsten Suel, and Nicola Tonellotto. 2021. Learning Passage Impacts for Inverted Indexes. Association for Computing Machinery, New York, NY, USA, 1723–1727. <https://doi.org/10.1145/3404835.3463030>
- [32] Pascal Massart. 1990. The tight constant in the Dvoretzky-Kiefer-Wolfowitz inequality. *The Annals of Probability* (1990), 1269–1283.
- [33] Bhaskar Mitra, Eric Nalisnick, Nick Craswell, and Rich Caruana. 2016. A dual embedding space model for document ranking. *arXiv preprint arXiv:1602.01137* (2016).
- [34] Bhaskar Mitra, Corby Rosset, David Hawking, Nick Craswell, Fernando Diaz, and Emine Yilmaz. 2019. Incorporating query term independence assumption for efficient retrieval and ranking using deep neural networks. *arXiv preprint arXiv:1907.03693* (2019).
- [35] Rodrigo Nogueira, Jimmy Lin, and AI Epistemic. 2019. From doc2query to docTTTTTquery. *Online preprint* (2019).
- [36] Prafull Prakash, Julian Killingback, and Hamed Zamani. 2021. Learning Robust Dense Retrieval Models from Incomplete Relevance Labels. In *Proceedings of the 44th International ACM SIGIR Conference on Research and Development in Information Retrieval*. 1728–1732.
- [37] Koustav Rudra and Avishek Anand. 2020. Distant Supervision in BERT-Based Adhoc Document Retrieval. In *Proceedings of the 29th ACM International Conference on Information & Knowledge Management (CIKM '20)*. Association for Computing Machinery, New York, NY, USA, 2197–2200. <https://doi.org/10.1145/3340531.3412124>
- [38] Koustav Rudra, Zeon Trevor Fernando, and Avishek Anand. 2021. An In-depth Analysis of Passage-Level Label Transfer for Contextual Document Ranking. *CoRR abs/2103.16669* (2021).
- [39] Shuai Wang, Shengyao Zhuang, and Guido Zuccon. 2021. BERT-based Dense Retrievers Require Interpolation with BM25 for Effective Passage Retrieval. In *Proceedings of The 11th International Conference on the Theory of Information Retrieval*.
- [40] Xing Wei and W. Bruce Croft. 2006. LDA-Based Document Models for Ad-Hoc Retrieval. In *Proceedings of the 29th Annual International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '06)*. Association for Computing Machinery, New York, NY, USA, 178–185. <https://doi.org/10.1145/1148170.1148204>
- [41] Lee Xiong, Chenyan Xiong, Ye Li, Kwok-Fung Tang, Jialin Liu, Paul N. Bennett, Junaid Ahmed, and Arnold Overwijk. 2021. Approximate Nearest Neighbor Negative Contrastive Learning for Dense Text Retrieval. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=zeFrfyZln>

- [42] Jingtao Zhan, Jiaxin Mao, Yiqun Liu, Jiafeng Guo, Min Zhang, and Shaoping Ma. 2021. *Optimizing Dense Retrieval Model Training with Hard Negatives*. Association for Computing Machinery, New York, NY, USA, 1503–1512. <https://doi.org/10.1145/3404835.3462880>
- [43] Shengyao Zhuang and Guido Zuccon. 2021. *TILDE: Term Independent Likelihood MoDEL for Passage Re-Ranking*. Association for Computing Machinery, New York, NY, USA, 1483–1492. <https://doi.org/10.1145/3404835.3462922>

ANCE	
Doc'19,	castorini/ance-msmarco-doc-maxp
Doc'20	msmarco-doc-ance-maxp-bf
PASSAGE'19	castorini/ance-msmarco-passage msmarco-passage-ance-bf
TCT-COLBERT	
Doc'19,	castorini/tct_colbert-msmarco
Doc'20	msmarco-doc-tct_colbert-bf
PASSAGE'19	castorini/tct_colbert-msmarco msmarco-passage-tct_colbert-bf

Table 5: The pre-trained dense encoders and corresponding indexes we used in our experiments. In each cell, the first line corresponds to a pre-trained encoder (to be obtained from the *HuggingFace Hub*) and the second line is a pre-built index provided by PYSERINI.

A EXPERIMENTAL DETAILS

In this section we provide details regarding our experiments to ensure reproducibility. This includes hardware, software, model hyperparameters as well as techniques employed.

A.1 Hardware Configuration and Latency Measurements

Our experiments are performed on a single machine using an Intel Xeon Silver 4210 CPU with 40 cores, 256GB of RAM and an NVIDIA Tesla V100 GPU. In order to measure the per-query latency numbers, we perform each experiment four times and report the average latency, excluding the first measurement (in order to account for any potential caching). In general, latency is reported as the sum of scoring (this includes operations like encoding queries and documents, obtaining representations from a FAST-FORWARD index, computing the scores as dot-products and so on), interpolation (cf. Equation (2)) and sorting cost. Any pre-processing or tokenization cost is ignored. Further, the first-stage (sparse) retrieval step is not included, as it is constant for all methods. The FAST-FORWARD indexes are loaded into the main memory entirely before they are accessed.

A.2 Software and Hyperparameters

We use the PYSERINI toolkit for all of our retrieval experiments, which uses the *HuggingFace transformers* library internally. PYSERINI provides a number of pre-trained encoders and corresponding indexes. Table 5 gives an overview over the ones we used for our experiments. Our dense encoders (ANCE and TCT-COLBERT) output 768-dimensional representations. Our sparse BM25 retriever is provided by PYSERINI as well. We use the pre-built indexes msmarco-passage ($k_1 = 0.82$, $b = 0.68$) and msmarco-doc ($k_1 = 4.46$, $b = 0.82$).